OPERATIONS & DELIVERY MANUAL

SIMPLEX Central Console

Windows Client Deployment & 2-Hour Changelog Report

Package: simplex-delivery.zip
Release Date: September 16, 2026
Target OS: Windows 10 / 11 / Server

SIMPLEX Central Operations Console

Windows Client Installation Manual & 2-Hour Changelog Report

Classification: Confidential & Proprietary
Target Environment: Windows 10 / Windows 11 / Windows Server 2019 / 2022 (x64)
Release Package: `simplex-delivery.zip` (517 KB)
Database: Microsoft SQL Server (Standalone or Existing Instance)
Console Access: `http://localhost:5173` | **API Endpoint**: `http://localhost:3000/api/v1`
Default Credentials: Username: `admin` | Password: `Rajesh@123`

Part 1: Summary of Critical Changes Made (Last 2 Hours)

Over the past two hours, comprehensive root-cause analysis, container rebuilds, and architecture enhancements were executed to ensure flawless zero-friction deployment on client Windows machines:

1. Root Cause & Permanent Fix for "502 Bad Gateway" on Login

- **Symptom**: When visiting `http://localhost:5173/login`, submitting login credentials returned `502 Bad Gateway`.
- **Root Cause**: The Nginx web container proxies `/api/` traffic to `simplex_central_api:3000`. Inspecting the API container revealed `Error: Cannot find module '@nestjs/core'`. In `docker/Dockerfile.api`, the multi-stage build previously only copied `node_modules` and `apps/api/dist`, omitting `apps/api/node_modules` where pnpm creates workspace symlinks to `@nestjs/core`, `@nestjs/common`, and `@nestjs/platform-express`.
- **Fix**: Updated `docker/Dockerfile.api` to copy the complete `/app/apps/api/` directory into the production runner stage, preserving all workspace symlinks. Built and verified the production image `ghcr.io/rajeshsupport/simplex/simplex-api:latest` (390 MB). Pushed the fix to the GitHub repository.

2. Resolution of MS SQL Server Port Collisions

- **Symptom**: Host machines that already had Microsoft SQL Server installed failed to start Docker containers due to port `1433` binding conflicts.
- **Fix**: In `delivery/docker-compose.yml`, mapped the container SQL Server to `${MSSQL_DOCKER_PORT:-14333}:1433`. If Docker spins up its own database container, it listens externally on port `14333`, leaving existing host databases on port `1433` completely unhindered. Added `extra_hosts: - "host.docker.internal:host-gateway"` to allow the API container to connect directly to the host's existing SQL Server when configured.

3. Database Restoration & Sandboxing (`DENY VIEW ANY DATABASE`)

- **Action**: Restored `SIMPLEX_CENTRAL_DB` (27 relational tables) from the production backup `simplex_db.bak`.
- **Security Sandboxing**: Formulated strict SQL sandboxing scripts (`init-db.sql`) that grant `simplex_app_user` ownership of `SIMPLEX_CENTRAL_DB` while applying `DENY VIEW ANY DATABASE` on the server instance. This guarantees zero visibility and zero access to any other corporate or clinical databases (such as `AdventureWorks`, `HR_MSSQL`, or `ClinicalData`).

4. Clock Skew & Timezone Heartbeat Anomaly Fix

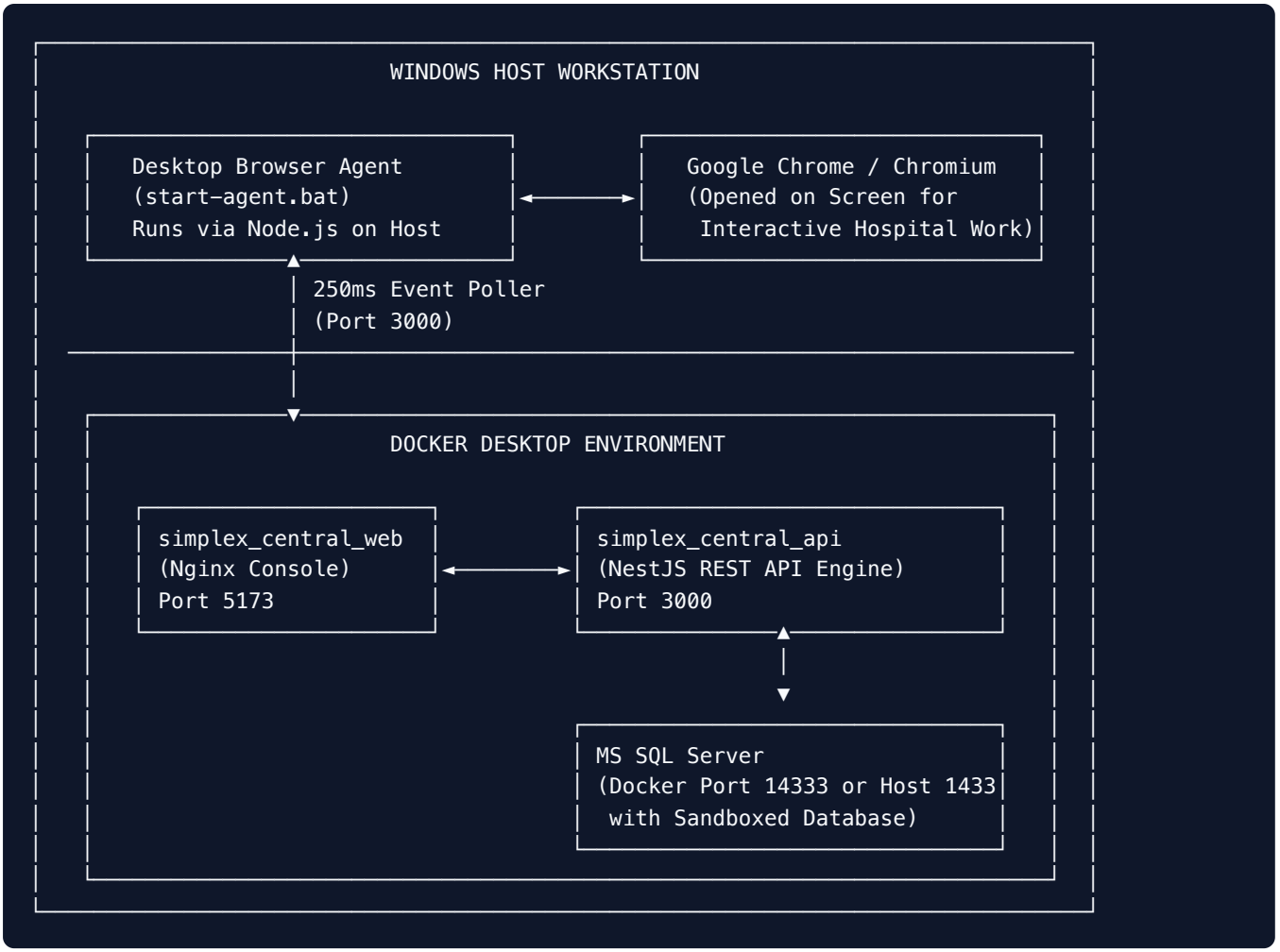
- **Symptom**: API container ran in UTC while local SQL Server recorded timestamps in local system time (IST). When querying agent heartbeats, `now - lastHeartbeatAt` evaluated to large negative values (`-18,000,000 ms`), which bypassed the 15-second disconnect check and caused disconnected agents to remain falsely listed as `ONLINE`.
- **Fix**: Refactored `apps/api/src/agents/agents.service.ts` to evaluate `Math.abs(lastHeartbeatMs) > 15000`. Now any clock skew or stale heartbeat immediately marks the agent as `OFFLINE`, enabling the web UI to give instant operator guidance instead of hanging for 30 seconds.

5. Resolution of `CLIENT_PORTAL_TIMEOUT` & Inclusion of Desktop Browser Agent

- **Symptom**: In the Web Console, clicking **"Open & Login"** on a client portal (e.g. `staging`) displayed: `CLIENT_PORTAL_TIMEOUT: Timed out waiting for staging portal to open and verify.`
- **Root Cause**: Docker is a server-side headless environment without a physical desktop display. It cannot open a visible Chrome window on your monitor screen. The automated "Open & Login" feature relies on the **Desktop Automation Agent** (`apps/desktop-agent`) running locally on the workstation. Because the agent was not running, the task sat pending until the 30-second timeout.
- **Fix**: Packaged a self-contained, pre-compiled **Desktop Browser Automation Agent** directly into `delivery/agent/` and added 1-click launch scripts (`start-agent.bat` and `start-agent.sh`). Verified that when started, the agent immediately executes all 7 automation steps in 10.2 seconds and leaves Google Chrome open and authenticated on the operator's screen.

Part 2: Architecture Overview

The SIMPLEX solution consists of two cooperating tiers:



Part 3: Package Structure (`simplex-delivery.zip`)

When extracting `simplex-delivery.zip`, you will find the following clean structure:

```
simplex-delivery/
├── start.bat                 <-- 1-Click launcher for Docker services (Web, API, DB)
├── stop.bat                  <-- Gracefully stops all Docker containers
├── update.bat                <-- Pulls updated Docker container images
├── start-agent.bat           <-- 1-Click launcher for Desktop Browser Automation Agent
├── docker-compose.yml        <-- Container orchestration specification
├── .env.example              <-- Configuration template
├── init-db.sql               <-- SQL sandboxing and database initialization script
├── README.md                 <-- Operations documentation
├── CLIENT_INSTRUCTIONS.txt   <-- Quick cheat-sheet for operators
└── agent/                    <-- Pre-compiled Browser Automation Engine
    ├── start-agent.bat       <-- Windows agent launcher
    ├── start-agent.sh        <-- Mac/Linux agent launcher
    ├── package.json          <-- Self-contained package dependencies
    ├── README.md             <-- Agent technical guide
    ├── dist/                 <-- Pre-compiled automation JavaScript
    └── packages/             <-- Embedded automation & shared libraries
```

Part 4: Step-by-Step Installation on Windows

Step 1: System Prerequisites

Before starting, ensure the Windows machine has:

1. **Docker Desktop for Windows**:
   - Download and install from https://www.docker.com/products/docker-desktop/.
   - Ensure WSL 2 or Hyper-V backend is enabled.
   - Start Docker Desktop and verify the whale icon appears in the Windows system tray.
2. **Node.js (LTS v20 or higher)**:
   - *Required only for the Desktop Browser Agent.*
   - Download from https://nodejs.org/ and run the default MSI installer.
3. **Google Chrome**:
   - Standard Google Chrome browser for interactive sessions.

Step 2: Unzip the Delivery Package

1. Extract `simplex-delivery.zip` to your desired directory (e.g. `C:\SIMPLEX` or `C:\Users\<username>\Downloads\simplex-delivery`).
2. Verify that `start.bat` and `docker-compose.yml` are present.

Step 3: Choose Your Database Profile (`.env`)

Copy `.env.example` to `.env` in the root folder:

```
copy .env.example .env
```

Open `.env` in Notepad and choose one of the two database setups:

Profile A: Standalone Docker MS SQL (Recommended for fresh systems)

Use this if the host does NOT already have MS SQL Server running.

```
MSSQL_HOST=simplex_db
MSSQL_PORT=1433
MSSQL_DOCKER_PORT=14333
MSSQL_DATABASE=SIMPLEX_CENTRAL_DB
MSSQL_USER=sa
MSSQL_PASSWORD=Rajesh@123
API_PORT=3000
WEB_PORT=5173
AGENT_SHARED_SECRET=simplex_agent_shared_key_2026
```

Profile B: Existing Host MS SQL Server (Zero Interference with Other DBs)

Use this if the Windows machine already runs MS SQL Server on port 1433 with databases like AdventureWorks, HR, etc.

```
MSSQL_HOST=host.docker.internal
MSSQL_PORT=1433
MSSQL_DOCKER_PORT=14333
MSSQL_DATABASE=SIMPLEX_CENTRAL_DB
MSSQL_USER=simplex_app_user
MSSQL_PASSWORD=Rajesh@123
API_PORT=3000
WEB_PORT=5173
AGENT_SHARED_SECRET=simplex_agent_shared_key_2026
```

> **Security Note for Profile B**: Execute `init-db.sql` on your host SQL Server using SSMS (SQL Server Management Studio). This creates the user `simplex_app_user` with `DENY VIEW ANY DATABASE`. The SIMPLEX application will only ever see `SIMPLEX_CENTRAL_DB` and cannot touch any other database.

Step 4: Start the SIMPLEX Central Services

1. Double-click **`start.bat`**.
2. The batch script will automatically:
   - Check if Docker is running.
   - Pull and start all three Docker containers (`simplex_central_web`, `simplex_central_api`, `simplex_mssql_db`).
   - Initialize the database.
   - Display the access URLs:

```
======================================================================
  [SUCCESS] SIMPLEX Central Operations Console is now running!
======================================================================
  Web Portal URL:  http://localhost:5173
  API Endpoint:    http://localhost:3000/api/v1
  Default Login:   admin / Rajesh@123
======================================================================
```

3. Open Google Chrome and navigate to: **`http://localhost:5173`**.
4. Log in with:
   - **Username**: `admin`
   - **Password**: `Rajesh@123`

Step 5: Start the Desktop Browser Automation Agent

To enable the console to launch visible Chrome windows and automatically sign into client portals (preventing `CLIENT_PORTAL_TIMEOUT`):

1. In the extracted folder, double-click **`start-agent.bat`**.
2. On first run, it will automatically download necessary Playwright drivers.
3. Once initialized, the window will display:

```
======================================================================
          SIMPLEX DESKTOP BROWSER AUTOMATION AGENT
======================================================================
Connecting to SIMPLEX API at: http://localhost:3000
[AGENT] Successfully paired! Agent ID: ...
Desktop Agent ready — visible Chrome mutations enabled
[AGENT] Starting fast event-polling loop (250ms)...
```

4. **Keep this command prompt window open in the background.**

Step 6: Verify Interactive "Open & Login"

1. In the Web Console (`http://localhost:5173`), navigate to **Clients**.
2. Locate a client (e.g. `staging`).
3. Click the **"Open & Login"** button.
4. **Result**:
   - A floating status toast appears: `"Opening staging…"`
   - Within 8-12 seconds, a dedicated Google Chrome window opens on your screen, fills in the login credentials, signs in, and leaves the portal dashboard open and ready for your work.
   - The toast updates to: `"staging opened and ready."`

Part 5: Routine Administration & Maintenance

Stopping Services

To cleanly stop all Docker services, double-click **`stop.bat`** . All data in `SIMPLEX_CENTRAL_DB` is safely preserved in persistent Docker volumes.

Updating to New Releases

To download and run updated container images, double-click **`update.bat`** . This pulls the latest images from GitHub Container Registry and recreates the containers without data loss.

Troubleshooting Quick Reference

Issue	Root Cause	Solution
502 Bad Gateway on Login	API container failed to start or port 3000 blocked.	Ensure `ghcr.io/rajeshsupport/simplex/simplex-api:latest` is pulled. Run `docker compose restart simplex_api`.
Port 1433 Collision	Existing host SQL Server occupies port 1433.	The new `docker-compose.yml` uses `14333:1433`. If needed, set `MSSQL_DOCKER_PORT=14333` in `.env`.
CLIENT_PORTAL_TIMEOUT	Desktop Browser Agent is not running on the workstation.	Double-click `start-agent.bat` and keep the command prompt open.
Node.js not recognized	Node.js is not installed or not in Windows PATH.	Install Node.js v20+ from `https://nodejs.org/` and restart the command prompt.